THE GAME — Adding Cases 2 to 7

Implementation guide for a coding agent (Codex)

0. Starting from a new session (read first)

You have no memory of earlier work. Everything you need is in this guide and the repository.

Project in one paragraph. THE GAME is a 3D investigation game for a live campus event. Teams sit at a 3D desk with seven case folders. Each case has 4–5 levels (puzzles), and solving a level pays score and coins. Teams spend coins on power-ups in a Normal Market and a Black Market. Solving a whole case pays digits of a code that unlocks an evidence phone. It is a **Next.js 16 + React 19 + TypeScript** app, with three.js for the 3D scenes. The game client is in `game/`, mounted by `app/[[...slug]]`. Server actions in `actions/game.ts` talk to **Supabase** (Postgres), where answers are checked by SQL functions (`supabase/migrations/004–006`).

Two modes, set by `NEXT_PUBLIC_GAME_BACKEND`:

Mode	Where it runs	How answers are handled
`mock`	Everything in the browser; no database needed	Answers are read from `secrets.ts`. Not secure; for development only.
`supabase`	The real event	`secrets.ts` is replaced by an empty stub at build time. Answers live only in the database.

`.env.development` already selects `mock`, so you can run and test without any keys.

Setup:

```
        git clone https://github.com/neilmalvalli-cyber/THE_GAME.git && cd THE_GAME   # branch main
        npm install            # Node 20+ (developed on Node 22)
        npm run dev            # mock mode on http://localhost:3000
```

Then:

1. Click **Team Login → Create Team** with any name and password.
2. Skip the intro.
3. Open a folder on the desk.

For the admin view (mock mode only), use **Admin Login** with the password from `.env.development`.

What you will NOT have, and must not ask for or invent:

- Supabase keys or the database password. The organizers run the seed SQL themselves (section 6).
- Real evidence files that are not in the brief.

Read before editing:

- `game/data/cases.ts`: all of Case 1. It is the reference.
- The top comment of `game/data/secrets.ts`.

Next.js note: this Next.js version differs from older ones (see `AGENTS.md`). You should not need to touch any Next.js code to add cases.

Testing in a container without a browser or Postgres:

- `npx tsc --noEmit` and `npm run seed` always work.
- `npm run test:e2e` needs Playwright with Chromium (`npx playwright install chromium` if it is missing) and the dev server running.
- `npm run test:sql` needs a local PostgreSQL 15+ (`PGHOST`, `PGPORT`, `PGUSER`; it creates its own `thegame_test` database). If Postgres is unavailable, say so in the report. Do not skip silently.

1. What this document is

This guide is for a coding agent working in the **THE_GAME** repository (`https://github.com/neilmalvalli-cyber/THE_GAME`, branch `main`). It explains how to add four new investigation cases to the game. **Case 1 (“The Fractured Truth”) is already finished. Use it as the reference implementation.**

Seven cases, two people working at the same time. The 3D desk shows one folder per case. The folder layout, camera framing and phone-code split are already set up for **seven cases**.

Case	Status now	Who fills it
Case 1	**Finished**. Reference implementation; do not edit	—
Case 2	Placeholder (“The Encrypted Trail”)	**Person A** (the organizer)
Case 3	Placeholder (“The Ghost File”)	**Person A**
Case 4	Placeholder	**Person B** (teammate)
Case 5	Placeholder	**Person B**
Case 6	Placeholder (“The Unknown File”)	**Person B**
Case 7	Placeholder (“The Seventh Seal”, 4 placeholder levels)	**Person A**

Each new case **replaces the placeholder in its slot**. Keep the slot’s `id`.

Do not add an 8th case. It would need a new phone-code split; see section 9.

Working in parallel without conflicts

Two Codex sessions edit the same two files (`game/data/cases.ts`, `game/data/secrets.ts`) at the same time. To keep merges clean:

1. **Edit only your own case blocks:**
   - in `cases.ts`, the `{ id: N, … }` object;
   - in `secrets.ts`, the `'N-…'` lines in `ANSWERS`, `HINTS` and `DEBRIEFS`.
2. **Do not reorder, reformat or "tidy" anything else** in those files. Do not run a formatter over the whole file.
3. **Do not change shared settings:**
   - `CASE_FRAGMENT_LENGTH`, `UNLOCK_CODE`, `LEVEL_SCORE`;
   - the interfaces at the top of `cases.ts`;
   - the test files.

   The tests already read case and level counts from the data, so no test edits are needed for new content.
4. **Before you start and before you push:** run `git pull --rebase origin main`. If the rebase reports a conflict in `cases.ts` or `secrets.ts`, keep **both** sides: the other person's case and yours. Then re-run `npx tsc --noEmit` and `npm run seed`.
5. **Commit one case per commit:** `Case N: <title> — <k> levels`.
6. **Seeding is shared.** Only one person (the organizer) runs the final seed in Supabase, after both sides are merged (section 6).

The human will give you the content of each case: titles, story text, evidence, answers and hints. Your job is to put that content into the data files, keep the answers secret, and verify everything. **The case pages, attempt counting, scoring, admin tools and database rules already exist and do not need new code.**

Paste this prompt into Codex together with this PDF

> Read the attached guide "THE GAME — Adding Cases 2 to 7" and follow it exactly, including "Working in parallel without conflicts". Another person is editing other cases at the same time. Implement the case brief(s) I give you below as Case N (replacing the placeholder in that slot); edit only that case. Use Case 1 in `game/data/cases.ts` as the reference. Put answers, hints and debriefs ONLY in `game/data/secrets.ts`. Do not change Case 1, the 3D scenes, the SQL migrations or the server code unless the guide says so. Use the content exactly as written; do not invent clues, evidence or answers — ask me if something is missing. When done, run every check in section 7 and report the results in the format of section 8.
>
> CASE BRIEF: (paste the filled template from section 10 here)

2. Non-negotiable rules

1. **Answers never reach the browser.** Answers, Powerful Hints and debriefs go **only** in `game/data/secrets.ts`.

    - Never put them in `cases.ts`, in components, in placeholders, in hidden inputs, in comments shown on screen, or in `public/`.
    - In the event build (`NEXT_PUBLIC_GAME_BACKEND=supabase`), `secrets.ts` is swapped for `secrets.stub.ts`, and the answers live only in the Supabase database.
2. **Do not touch Case 1** (`id: 1`) or its secrets.
3. **Do not fabricate content.** If the brief leaves something out, ask the human. That includes:
    - an evidence file, image, video, link or table;
    - an answer;
    - a hint.

   If an optional video URL is unknown, leave `url: ''`; the page then hides it.
4. **Do not reveal physical locations or coordinates** that teams are supposed to discover in the real world. Organizer-only notes stay out of the repository's player-facing text.
5. **No sequential unlocking.** Every level of a case is open from the start. Levels are independent.
6. **No secrets in git.** Never commit any of these:
    - `.env*` files with real keys;
    - `supabase/seed/the_game_seed.sql`, which is git-ignored because it contains the answers;
    - any Supabase key or password.
7. **Keep it data-driven.** Do not create a standalone app or new pages per case. A case is data in two files.

3. Where things live

File	What it holds	Visible to players?
`game/data/cases.ts`	Case and level content: titles, story sections, tables, links, answer format, attempts, score, coins, success text	Yes (public)
`game/data/secrets.ts`	`ANSWERS`, `HINTS` (Powerful Hints), `DEBRIEFS`, `UNLOCK_CODE` (phone code)	**No** (server/database only)
`game/data/secrets.stub.ts`	Empty stand-in used by the event build	—
`public/evidence/...`	Images or files for `evidenceAssets` (create the folder if needed)	Yes (anyone can download)
`scripts/generate-seed.mjs`	`npm run seed`: turns cases.ts + secrets.ts + market.ts into the database seed SQL	—
`supabase/migrations/004–006`	Database tables and rules (attempt limits, answer formats, admin controls). **Already done; do not edit.**	—
`game/pages/LevelPage.tsx`, `DeskPages.tsx`	Renders any case from the data. No changes needed.	—

Anything in `public/` is downloadable by anyone. Never put the answer in an evidence file's name or contents unless the puzzle is meant to be solved from it.

4. The data format

4.1 A case (`CaseDef` in `game/data/cases.ts`)

```
        {
          id: 2,                                  // slot 2–7; keep the id of the
placeholder you replace
          title: 'The Case Title',                // printed on the 3D folder cover —
keep it short (≤ 28 chars)
          classification: 'RESTRICTED',           // optional label on the case
overview
          closedLines: ['ALL FOUR INVESTIGATIONS VERIFIED', 'THE TRUTH HAS BEEN
RECOVERED', 'CASE STATUS: COMPLETED'],
                                                  // optional: lines on the CASE FILE
CLOSED screen
          levels: [ /* 4 or 5 LevelDef objects, numbered 1..n */ ],
        }
```

Number of levels. Use **4 or 5** per case. The desk has photo layouts for 4 and 5 (`LAYOUTS` in `game/scenes/desk/engine/scene/PolaroidEvidence.js`). Any other count needs a new layout

entry there; add one only if the brief requires it.

Case completion. A case is complete when **all** its levels are completed, in any order. Skip Tokens and Master Keys count towards completion.

4.2 A level (`LevelDef`)

```
        {
          number: 1,                                  // 1..n, unique within the case
          title: 'Level Title',
          difficulty: 'Easy',                         // 'Easy' | 'Medium' | 'Hard'
          reward: 100,                                // coins paid once on completion
          score: 100,                                 // leaderboard points paid once on
completion
          maxAttempts: 8,                             // wrong answers before the level
locks (omit = unlimited)
          classification: 'FIELD INVESTIGATION',      // label in the level header
          description: 'One or two sentences shown on the typed report inside the 3D
folder.',
          briefing: 'One-line objective (fallback text).',
          evidence: '',                               // legacy field: keep '' when you
use sections
          sections: [ /* rich page content, see 4.3 */ ],
          question: 'What is the name of …?',         // optional question above the
answer box
          answerType: 'text',                         // see 4.4
          placeholder: 'ENTER KEYWORD',
          submitLabel: 'SUBMIT FINAL ANSWER',
          formatHint: 'Example format: …',            // optional; must NOT give the
answer away
          successLines: ['EVIDENCE VERIFIED', 'LEVEL 01 — COMPLETED'],
        }
```

4.3 Page sections (`LevelSection`)

A level page is a list of sections. Each one can use any of these fields:

Field	Renders as
`heading`	Section title
`body: string[]`	Paragraphs
`document: string[]`	A “classified document” box (quoted clue, intelligence report)
`list: string[]`, `ordered: true`	Bullet or numbered list
`code: string`	A monospaced line (system status, log line)
`tone: 'warning' | 'note'`	Red warning box or grey note box
`table: { title, subtitle, columns, rows }`	Evidence table (rows are arrays of strings)
`link: { label, url }`	External evidence button that opens in a new tab (e.g. a Google Drive folder)
`video: { title, url }`	Embedded video player for a **direct video file URL** (e.g. `/evidence/case-02/clip.mp4` or a direct .mp4 link). `url: ''` hides it. A YouTube or Drive *page* link will not play: use a `link` section for those.

For a full example of every section type, see Case 1 levels 3 (table + warning), 4 (Drive link) and 5 (video + witness cards) in `cases.ts`.

4.4 Answer types and how answers are checked

`answerType`	Player sees	Answer format in `secrets.ts`	Accepted
`'text'` (default)	One text box	`'cv raman'`	Case, spaces and punctuation ignored: `C.V. Raman`, `cv raman` and `CVRAMAN` all match
`'numeric'`	Number box	`'2'`	As text, plus leading zeros ignored (`02` = `2`)
`'choice'`	Radio buttons from `choices: ['A','B',…]`	`'B'`	The chosen option
`'fields'`	Several boxes, `fields: [{key,label,placeholder}, …]`, submitted together as **one** attempt	`'part1|part2|part3'`, the same number of parts as fields	Every part must match (each normalised as text). A time part may omit “UTC”.
`'witness'`	Witness cards from `choices: ['1','2','3','4']`	`'2'`	`2`, `witness 2`, `Witness 02`

Important:

- **One answer per level.** Alternative spellings are not supported unless the normalisation above covers them. If the brief lists several accepted answers that normalisation does not cover, ask the human which one to use.
- **No other answer types.** If a brief needs something new (for example “any of these three words”), stop and ask. Do not improvise. It would need matching changes in both:
  - `game/services/levels.ts` (mock build), and
  - a new SQL migration (`game_answer_ok`).

4.5 Secrets (`game/data/secrets.ts`)

Keys are `'<case>-<level>'`. **Replace the placeholder entries for the slot you fill. Never leave old placeholder answers behind.**

```
        export const ANSWERS = {
          // Case 2 — NEW TITLE
          '2-1': 'answer one',
          '2-2': 'answer two',
          '2-3': '42',
          '2-4': 'account|v2|10:15:00 UTC',   // 'fields' level: parts in the same order as `fields`
          '2-5': '3',                         // only if the case has 5 levels
        };
        export const HINTS = {
          '2-1': 'A Powerful Hint teaches the METHOD, never the answer.',
          // one per level
        };
        export const DEBRIEFS = {
          '2-4': 'Shown to the team ONLY after they solve level 2-4 (e.g. a verification summary).',
        };
```

- **Hints** are bought with coins (Powerful Hint power-up). Use the hint text from the brief. If the brief has none, ask the human. Never write a hint that contains the answer.
- `UNLOCK_CODE` and `CASE_FRAGMENT_LENGTH` (the phone code digits each solved case pays out) **do not change**.

4.6 Score and coins

- Use the values in the brief.
- If the brief gives none, follow Case 1’s scale: Easy 100, Medium 150, Hard 200–250 for both score and coins.
- Points and coins are paid **once** per level per team. Wrong answers cost nothing except an attempt.

5. Step by step for one case

1. Read the brief. List every level with its answer type, answer, attempts and any links, tables or videos.
2. In `game/data/cases.ts`, replace the placeholder object for that slot. Keep the `id`. Write the levels with the exact brief text in `sections`.
3. In `game/data/secrets.ts`, replace that slot’s `ANSWERS` and `HINTS`, and add any `DEBRIEFS`.

4. Put evidence images or files under `public/evidence/case-0N/` and list them in `evidenceAssets`. Do this only if the brief supplies the files.
5. Run `npm run seed`. It must print `6 cases, N levels` with no “Cannot write the seed” errors. It checks that:
    - every level has an answer;
    - every `fields` answer has the right number of parts.
6. Run the checks in section 7 and fix anything that fails.
7. Commit with a clear message, for example `Case 2: <title> — <n> levels`, and push to `main`. **Do not commit** `supabase/seed/the_game_seed.sql`**.**

6. Putting new cases live (the human does this; include it in your report)

1. On the host PC: `git pull`.
2. Run `npm run seed`. This writes `supabase/seed/the_game_seed.sql` from the local `secrets.ts`.
3. In the Supabase dashboard, open **SQL Editor**, paste the whole seed file and run it. This is safe to re-run: it updates the cases in place and keeps teams’ progress.
4. Restart the server: `npm run lan`.

No new migration is needed for new cases. Migrations 004, 005 and 006 must already have been run once.

Changing a level that teams have already played (for example its answer) does not reset their attempts. Use the Admin page instead: **Teams → team → Reset attempts / Reopen / Mark completed**.

7. Checks to run before you finish

#	Command / action	Expected
1	`npx tsc --noEmit`	No errors
2	`npm run seed`	`Wrote …: 6 cases, N levels …`, no problems listed
3	`npm run test:sql` (needs a local Postgres; set `PGHOST` / `PGPORT` / `PGUSER`)	All PASS, including the 2 concurrency checks
4	`npm run dev` (mock mode from `.env.development`), then `npm run test:e2e`	All checks pass (see 7.1)
5	Manual: open `/team/case/N`	The new title is on the folder; the overview lists every level as NOT STARTED with “8 / 8” attempts
6	Manual: each level page	Content matches the brief word for word; links open; no answer visible anywhere
7	Manual: wrong answer	“INCORRECT ANSWER — 7 ATTEMPTS REMAINING”; after 8 wrong answers the level shows ATTEMPTS EXHAUSTED and the other levels are unaffected
8	Manual: solve every level in a random order	CASE FILE CLOSED screen; a phone-code digit is awarded
9	Search the production build for answers: `NEXT_PUBLIC_GAME_BACKEND=supabase npx next build`, then `grep -ri "<an answer>" .next/static`	No matches

7.1 Tests and case layout

The tests read case and level counts from the data. **New content needs no test edits**, and you must not edit test files; the other person relies on them too. Things to know:

- `scripts/e2e.mjs` expects **7 cases**, fragments adding up to 9, and solves **Case 6** in mock mode with its answers from `secrets.ts`, so Case 6’s answers must be correct.
- `supabase/tests/game_test.sql` plays through **Case 2** in its generic checks (wrong answer, correct answer, skip token, solving every level). It reads the answers and level count from the database, so any valid Case 2 works.
- If a test fails because of a case you did not write, report it; do not “fix” the other person’s case.

Never weaken or delete a test to make it pass. Update the expectations to the new layout and explain why in the commit message.

8. Report format (what to send back)

1. **Cases added:** slot, title, levels (number, title, answer type, attempts, score/coins).
2. **Files changed.**
3. **Content questions:** anything missing from the brief that you did not invent.
4. **Check results:** each item of section 7 with PASS/FAIL and counts.
5. **Go-live steps for the organizers** (section 6), including which seed to run.
6. **Organizer-only notes:** physical locations, QR codes and coordinates. These belong in the report, never on the site.

9. Out of scope unless explicitly requested

- **More than seven cases.** This needs:
  - a check of the 3D desk framing (`game/scenes/desk/engine/config.js`; the layout itself is automatic);
  - a new phone-code split (`CASE_FRAGMENT_LENGTH` must still add up to the length of `UNLOCK_CODE`);
  - UI and test updates.
- **Changes to scoring rules, power-ups, markets, the phone, the admin page or SQL migrations.**
- **Changing Case 1.**

10. Case brief template (the human fills one per case)

```
CASE SLOT: 2 | 3 | 4 | 5 | 6 | 7
CASE TITLE (≤ 28 characters):
CLASSIFICATION (optional, e.g. RESTRICTED):
CASE CLOSED LINES (optional, up to 3):
NUMBER OF LEVELS: 4 | 5

--- LEVEL n ---
TITLE:
DIFFICULTY: Easy | Medium | Hard
SCORE / COINS:
MAX WRONG ATTEMPTS (default 8):
CLASSIFICATION LABEL (optional):
SHORT DESCRIPTION (1–2 sentences, shown inside the 3D folder):
PAGE CONTENT (exact text; mark headings, documents, tables, warnings, links, videos):
QUESTION (optional):
ANSWER TYPE: text | numeric | choice | fields | witness
  (fields → list each field label + placeholder in order)
  (choice / witness → list the options)
ANSWER (server-only):
POWERFUL HINT (method, not the answer):
DEBRIEF AFTER SOLVING (optional):
SUCCESS LINES (2 lines, e.g. "EVIDENCE VERIFIED" / "LEVEL 0n — COMPLETED"):
PLACEHOLDER / BUTTON TEXT (optional):
ORGANIZER-ONLY NOTES (locations, QR codes — never shown on the site):
```